

Task 2 — Hazelcast Distributed Data Structures

GitHub: https://github.com/Romko-studik/apz_homework_2/tree/hazelcast_basics

3. Distributed Map

3.1 Writing 1000 entries

Management Center — key distribution across 3 nodes:

Member	Entries	Gets	Puts	Removals	Sets	Entry Memory	Events	Hits
172.19.0.2:5701	322	0	0	0	1,000	42.69 kB	0	20
172.19.0.3:5701	340	0	0	0	0	45.11 kB	0	9
172.19.0.4:5701	338	0	48	0	0	44.80 kB	0	19
TOTAL	1,000	0	48	0	1,000	132.60 kB	0	48

1 - 3 of 3 Rows 10

3.2 Kill one node (docker stop hz3)

Management Center — distribution after killing hz3:

Map Statistics (In-Memory Format: BINARY) RESET TIME 1 minute ago → now Default View 🗑️ 🔗 +

Member	Entries	Gets	Puts	Removals	Sets	Entry Memory	Events	Hits
172.19.0.2:5701	486	0	0	0	1,000	64.45 kB	0	23
172.19.0.4:5701	514	0	48	0	0	68.15 kB	0	25
TOTAL	1,000	0	48	0	1,000	132.60 kB	0	48

1 - 2 of 2 Rows 10

3.3 Kill two nodes sequentially

Management Center — distribution after relaunching hz3:

Member	Entries	Gets	Puts	Removals	Sets	Entry Memory	Events	Hits
172.19.0.2:5701	322	0	0	0	1,000	42.71 kB	0	14
172.19.0.3:5701	337	0	0	0	0	44.67 kB	0	16
172.19.0.4:5701	341	0	48	0	0	45.22 kB	0	18
TOTAL	1,000	0	48	0	1,000	132.60 kB	0	48

Management Center — distribution after killing hz3, then hz2:

Member	Entries	Gets	Puts	Removals	Sets	Entry Memory	Events	Hits
172.19.0.4:5701	1,000	0	48	0	0	132.60 kB	0	48
TOTAL	1,000	0	48	0	0	132.60 kB	0	48

3.4 Kill two nodes simultaneously

Management Center — distribution after killing hz2 and hz3 at the same time:

Member	Entries	Gets	Puts	Removals	Sets	Entry Memory	Events	Hits
172.19.0.4:5701	688	0	48	0	0	91.22 kB	0	35
TOTAL	688	0	48	0	0	91.22 kB	0	35

3.5 Data loss analysis

We see that if we stop 1 or 2 nodes sequentially there is no data loss. However if 2 nodes are stopped at the same time we lose some entries - we have 688 entries remaining out of 1000 initial. This can be prevented by increasing backup-count in hazelcast settings, as this way each entry will exist on more nodes. So if we increase backup-count to 2 each entry will have a backup the 2 remaining nodes. This way if we lose 2 out of 3 nodes simultaneously we still don't lose any data

4. Distributed Map — No Locks

Console output from all 3 clients:

```
PS D:\UNI_UCU\APZ\apz_homework_2> python client/2_no_lock.py 1
>>
[Client 1] Starting 10K increments without locks...
[Client 1] Done. Current value: 11308

PS D:\UNI_UCU\APZ\apz_homework_2> python client/2_no_lock.py 2
>>
[Client 2] Starting 10K increments without locks...
[Client 2] Done. Current value: 13337

PS D:\UNI_UCU\APZ\apz_homework_2> python client/2_no_lock.py 3
>>
[Client 3] Starting 10K increments without locks...
[Client 3] Done. Current value: 14133
```

We can clearly see that the final value is much smaller than 30000 - due to race condition.

5. Pessimistic Locking

Console output from all 3 clients (final value should be exactly 30K):

```
PS D:\UNI_UCU\APZ\apz_homework_2> python .\client\3_pessimistic_lock.py 1
[Client 1] Starting 10K increments with pessimistic locking...
[Client 1] Reached 1/100th of increments...
[Client 1] Done in 161.95s. Current value: 29932

PS D:\UNI_UCU\APZ\apz_homework_2> python .\client\3_pessimistic_lock.py 2
[Client 2] Starting 10K increments with pessimistic locking...
[Client 2] Reached 1/100th of increments...
[Client 2] Done in 161.91s. Current value: 29978

PS D:\UNI_UCU\APZ\apz_homework_2> python .\client\3_pessimistic_lock.py 3
[Client 3] Starting 10K increments with pessimistic locking...
[Client 3] Reached 1/100th of increments...
[Client 3] Done in 161.68s. Current value: 30000
```

6. Optimistic Locking

Console output from all 3 clients (final value should be exactly 30K):

```

PS D:\UNI_UCU\APZ\apz_homework_2> python .\client\4_optimistic_lock.py 1
[Client 1] Starting 10K increments with optimistic locking...
[Client 1] Reached 100/10000 increments...
[Client 1] Done in 176.46s. Current value: 29906

PS D:\UNI_UCU\APZ\apz_homework_2> python .\client\4_optimistic_lock.py 2
[Client 2] Starting 10K increments with optimistic locking...
[Client 2] Reached 100/10000 increments...
[Client 2] Done in 176.41s. Current value: 29974

PS D:\UNI_UCU\APZ\apz_homework_2> python .\client\4_optimistic_lock.py 3
[Client 3] Starting 10K increments with optimistic locking...
[Client 3] Reached 100/10000 increments...
[Client 3] Done in 176.14s. Current value: 30000

```

7. Comparison

From the results of these 3 experiments we can clearly see that:

- Without locking we get race conditions resulting in loss of data - due to requests being skipped
- With both optimistic and pessimistic locking we get correct end value of 30000, however pessimistic locking works slightly faster

8. Bounded Queue

Writer + 2 readers console output:

```

[Reader 1] Got: 61
[Reader 1] Got: 63
[Reader 1] Got: 65
[Reader 1] Got: 67
[Reader 1] Got: 69
[Reader 1] Got: 71
[Reader 1] Got: 73
[Reader 1] Got: 75
[Reader 1] Got: 77
[Reader 1] Got: 79
[Reader 1] Got: 81
[Reader 1] Got: 83
[Reader 1] Got: 85
[Reader 1] Got: 87
[Reader 1] Got: 89
[Reader 1] Got: 91
[Reader 1] Got: 93
[Reader 1] Got: 95
[Reader 1] Got: 97
[Reader 1] Got: 99
[Reader 1] No more values, exiting.
[Reader 1] Read 50 values total.

[Reader 2] Got: 62
[Reader 2] Got: 64
[Reader 2] Got: 66
[Reader 2] Got: 68
[Reader 2] Got: 70
[Reader 2] Got: 72
[Reader 2] Got: 74
[Reader 2] Got: 76
[Reader 2] Got: 78
[Reader 2] Got: 80
[Reader 2] Got: 82
[Reader 2] Got: 84
[Reader 2] Got: 86
[Reader 2] Got: 88
[Reader 2] Got: 90
[Reader 2] Got: 92
[Reader 2] Got: 94
[Reader 2] Got: 96
[Reader 2] Got: 98
[Reader 2] Got: 100
[Reader 2] No more values, exiting.
[Reader 2] Read 50 values total.

[Writer] Put 80, queue size: 0
[Writer] Put 81, queue size: 0
[Writer] Put 82, queue size: 0
[Writer] Put 83, queue size: 0
[Writer] Put 84, queue size: 0
[Writer] Put 85, queue size: 0
[Writer] Put 86, queue size: 0
[Writer] Put 87, queue size: 0
[Writer] Put 88, queue size: 0
[Writer] Put 89, queue size: 0
[Writer] Put 90, queue size: 0
[Writer] Put 91, queue size: 0
[Writer] Put 92, queue size: 0
[Writer] Put 93, queue size: 0
[Writer] Put 94, queue size: 0
[Writer] Put 95, queue size: 0
[Writer] Put 96, queue size: 0
[Writer] Put 97, queue size: 0
[Writer] Put 98, queue size: 0
[Writer] Put 99, queue size: 0
[Writer] Put 100, queue size: 0
[Writer] Done.

```

We see that both readers perfectly split the entries, each reading one after another

Writer blocking when queue is full and no readers:

```
PS D:\UNI_UCU\APZ\apz_homework_2> python client/5_queue.py writer
[Writer] Writing values 1..100 to queue (max size 10)...
[Writer] Put 1, queue size: 1
[Writer] Put 2, queue size: 2
[Writer] Put 3, queue size: 3
[Writer] Put 4, queue size: 4
[Writer] Put 5, queue size: 5
[Writer] Put 6, queue size: 6
[Writer] Put 7, queue size: 7
[Writer] Put 8, queue size: 8
[Writer] Put 9, queue size: 9
[Writer] Put 10, queue size: 10
```

We see that

as expected - after we put 10 and reach the max queue size we are stopped, and since nothing is reading from it it just waits indefinitely.